

Asynchronous Programming

Presentation 6 Minutes - ملخص المحتوى + السكريبت

الفكرة العامة

ال Asynchronous Programming هي Programming Paradigm وليست Programming Language أو Tool. هي Concept بنستخدمه للتعامل مع العمليات التي تستغرق وقتًا بدون ما نخلي التطبيق يفضل Blocked لحد ما العملية تخلص.

1. المقدمة وال Story

نبدأ من تجربة بسيطة: نفتح Application مثل Facebook ونضغط على زر Comments يحتاج بيانات من السيرفر.

الكلام المقترح شفويًا

السلام عليكم، أنا ندى يسري. النهاردة هنتكلم عن توبيك مهم جدًا اسمه Asynchronous Programming. وهي Programming Paradigm، مش Programming Language ولا Tool. عشان نفهمها بسهولة، خلينا نبدأ بـ Story صغيرة.

2. المشكلة: التطبيق يعمل Freeze

تخيلي إن أول ما نضغط Comments، الشاشة كلها تجمد لحد ما السيرفر يرجع ال Response. المستخدم لن يستطيع الضغط أو ال Scroll، وغالبًا سيغلق التطبيق.

الكلام المقترح شفويًا

تخلوا إننا ضغطنا على زرار ال Comments، لكن بدل ال Shimmer أو Loading، التطبيق كله عمل Freeze. فضلنا ندوس ونحاول نتحرك، ومفيش حاجة بتستجيب. ليه؟ لأن البرنامج واقف مستني ال Response من السيرفر. وده سلوك سيئ جدًا وتجربة مستخدم مزعجة.

3. المشكلة أكبر من الـ Network فقط

الـ Network Request مجرد مثال. توجد أيضًا Database Queries و File Operations و Timers، وكلها قد تحتاج وقتًا.

الكلام المقترح شفويًا

الـ Network Request مش الحالة الوحيدة. عندنا كمان Database Queries، File Operations و Timers. كل دي عمليات ممكن تاخذ وقت، ومش منطقي إن الـ Application يفضل متجمد لحد ما تخلص.

4. الحل: Asynchronous Programming

بدل ما التطبيق يتوقف انتظارًا، يبدأ العملية ويكمل شغلًا آخر. عندما تنتهي العملية، ترجع النتيجة وتعامل معها.

الكلام المقترح شفويًا

طبيب إزاي نحل المشكلة دي؟ هنا بييجي دور ال Asynchronous Programming. البرنامج يبدأ العملية، لكن ما يفضلش واقف عليها. يكمل شغله بشكل طبيعي، ولما العملية تخلص والنتيجة ترجع، يتعامل معها ويكمل ال flow.

How It Works .5

Start Task → Async Operation → Continue Other Work → Result Returns → Handle Result
Async لا تعني عدم وجود Waiting؛ العملية نفسها ما زالت تحتاج وقتًا، لكن التنفيذ الرئيسي لا يظل Blocked بسببها.

الكلام المقترح شفويًا

الفكرة ببساطة: نبدأ ال Task، نسيبها لتنفيذ بشكل مستقل، والبرنامج يكمل شغل ثاني. لما ال Task تخلص، النتيجة ترجع ونبدأ نتعامل معاها. يعني ال Waiting موجود، لكن ال Blocking غير ضروري.

6. طرق التعامل: Async/Await → Promises/Futures → Callbacks

Callbacks: Function تنفذ عند انتهاء العملية. Promises/Futures: تمثل نتيجة ستصبح متاحة لاحقًا.

Async/Await: أسلوب أوضح وأسهل للقراءة.

الكلام المقترح شفويًا

طبيب كمبرمجين نعرف منين إن العملية خلصت وناخد نتيجتها إزاي؟ ظهر أكثر من approach: Callbacks، وبعدها Promises أو Futures، وأخيرًا Async/Await. الهدف إن التعامل مع الـ async code يبقى أوضح وأسهل.

7. Callback Hell

لما العمليات تعتمد على بعضها، ال Callbacks يمكن أن تتداخل داخل بعضها، فيصبح الكود أصعب في القراءة والصيانة.

الكلام المقترح شفويًا

لو عندي أكثر من عملية وكل واحدة مستتية نتيجة اللي قبلها، ال Callbacks تبدأ تدخل جوه بعض. كل ما ال steps more nested الكود يبقى وأصعب في القراءة والصيانة. وده اللي بنسميه Callback Hell.

مثال الكود

```
login(function(user) {
  getProfile(user, function(profile) {
    getPosts(profile, function(posts) {
      getComments(posts, function(comments) {
        // ...
      });
    });
  });
});
```

8. Async/Await Example

الـ async يجعل الـ function asynchronous وتسمح باستخدام الـ await. الـ await توقف تنفيذ هذه الـ function مؤقتًا حتى تصبح النتيجة جاهزة، بدون Blocking لباقي البرنامج.

الكلام المقترح شفويًا

خلينا نشوف مثال بسيط. عندنا function بتجيب User من API. استخدمنا async مع الـ function، و الـ await. عشان ننتظر نتيجة الـ fetch قبل ما نكمل الخطوة اللي بعدها. المهم إن الـ application كله مش لازم يقف.

مثال الكود

```
async function getUser(id) {  
  const response = await fetch(`/api/users/${id}`);  
  const user = await response.json();  
  return user;  
}
```

9. Async ≠ Parallel

Asynchronous: التركيز على عدم ال Blocking أثناء الانتظار. Parallelism: أكثر من عملية تُنفذ فعليًا في نفس الوقت.

الكلام المقترح شفويًا

هنا نقطة مهمة: Async مش معناها Parallel. ال Async بتتكلم عن إدارة وقت الانتظار وعدم ال Blocking، بينما Parallelism بتتكلم عن تنفيذ أكثر من operation فعليًا في نفس الوقت.

10. لماذا هي مهمة؟ + الخلاصة

تحافظ على Responsive UI، تستغل وقت الانتظار، تساعد التطبيقات على التعامل مع عمليات كثيرة، وتظهر في APIs و Databases و File I/O و Networking.

الكلام المقترح شفويًا

عشان كده بنشوف ال Asynchronous Programming في التطبيقات الحديثة طول الوقت. ال UI يفضل responsive، ووقت الانتظار ما يضيعش، والتطبيق يقدر يتعامل مع عمليات كثيرة بكفاءة. والخلاصة: نبدأ المهمة، نكمل شغل ثاني، ولما النتيجة تجهز نتعامل معاها.

توزيع وقت مقترح لـ 6 دقائق

- المقدمة والـ Story: 30-40 ثانية
- المشكلة: 45-60 ثانية
- الحل والـ How It Works: 60-75 ثانية
- 40 Callbacks / Callback Hell: ثانية
- Async/Await: 45-60 ثانية
- Async $\neq$ Parallel: 30 ثانية
- Why It Matters + الخلاصة: 40-50 ثانية